

ROS2

DAY 6 – Manual 1

Parameters – Fundamentals

1. What Exactly Is a Parameter?

A parameter is a named variable stored inside a node.

Simple analogy:

A ROS2 node is like a robot object, and parameters are its settings.

Example settings:

- robot speed
- lidar frame rate
- camera exposure
- PID gains
- sensor limits

! Key point:

Parameters exist inside the node, not globally.

2. Why Do We Even Need Parameters?

Without parameters ☹

You hardcode values:

```
speed = 2.0
```

To change speed → edit code → rebuild → restart node.

With parameters ✓

You declare:

```
self.declare_parameter("speed", 2.0)
```

Then change anytime:

```
ros2 param set /robot_node speed 5.0
```

- ✓ No code change
- ✓ No rebuild
- ✓ No restart

This is why every real robot uses parameters.

3. What Can You Store in a Parameter?

ROS2 parameters support types:

Type	Example
integer	10
double	1.5
bool	true
string	"hello"
array	[1, 2, 3]

No other types allowed.

Arrays must contain the same type (all int or all double, etc.).

4. How Parameters Actually Work (Internal Logic)

Every parameter has 3 parts:

1. Name → "speed"
2. Type → double
3. Value → 1.0

Stored inside a ROS2 parameter server built into the node.

That's why you can do:

```
ros2 param get /robot_node speed
```

ROS2 CLI contacts the node → node returns parameter.

5. Declaring a Parameter

Every parameter must be declared before use.

Why?

Because ROS2 must know:

- what type it is
- what its default value is
- that the parameter exists

Declaration:

```
self.declare_parameter("speed", 1.0)
```

Meaning:

Inside this node, create a parameter named "speed"

Its type is double

Default value is 1.0

If you try using a parameter without declaring → ROS2 ERROR

6. Accessing a Parameter (Getting Its Value)

After declaring:

```
speed = self.get_parameter("speed").value
```

The `.value` is important.

`get_parameter()` returns a `Parameter` object.

`.value` extracts the actual number/string.

7. Changing Parameters from Terminal

✓ Set a parameter:

```
ros2 param set /node_name param_name value
```

Example:

```
ros2 param set /robot_node speed 5.5
```

✓ Get a parameter:

```
ros2 param get /robot_node speed
```

✓ List all parameters:

```
ros2 param list
```

8. Static Parameters vs Dynamic Parameters

□ Static Parameter

- Value is read only once (at startup)
- Changing it does NOT update node behavior automatically

Example:

- Using parameter only in `__init__`

🔥 Dynamic Parameter

- Node reacts immediately when parameter changes
- Requires a parameter callback

Example:

- Live PID tuning
- Live speed control
- Live sensor threshold updates

Parameter callback is covered in Manual 2.

9. Now the Important Part: YAML Parameter Files

★ [What is a YAML File?](#)

YAML = "Yet Another Markup Language"

A YAML file is:

- a plain text file
- that stores configuration
- in key-value format
- like JSON but simpler

ROS2 uses YAML to store parameters for nodes.

10. ROS2 YAML Parameter File – Structure Explained

Here is a correct YAML file:

```
robot_node:
  ros_parameters:
    speed: 1.5
    max_distance: 10
    use_safety: true
    robot_name: "MY_ROBOT"
```

```
gains: [1.0, 0.5, 0.1]
```

Now let me break this down perfectly:

◆ Level 1 — Node name

```
robot_node:
```

Means:

These parameters belong to node named `/robot_node`.

◆ Level 2 — `ros__parameters`

```
ros__parameters:
```

Means:

Everything below this line is a ROS2 parameter.

◆ Level 3 — Parameters

```
speed: 1.5
max_distance: 10
use_safety: true
robot_name: "MY_ROBOT"
gains: [1.0, 0.5, 0.1]
```

These behave exactly like `declare_parameter()` values.

11. Using the YAML File in Code

Your node must declare parameters:

```
self.declare_parameter("speed")
self.declare_parameter("max_distance")
self.declare_parameter("use_safety")
self.declare_parameter("robot_name")
self.declare_parameter("gains")
```

12. Loading YAML File at Runtime

```
ros2 run my_pkg robot_node --ros-args --params-file params.yaml
```

Meaning:

Load params.yaml, feed values into the node.

13. Loading YAML in Launch File

In a .launch.py:

```
Node(  
    package='my_pkg',  
    executable='robot_node',  
    parameters=['params.yaml']  
)
```

Now the launch file provides parameters to the node automatically.

14. Summary

Concept	Meaning
Parameter	A named variable inside a node
Declare	Register parameter & set default
Get	Access value inside code
Set	Change value at runtime
Dynamic	Node reacts to changes immediately
YAML	External configuration file for parameters
Launch File	Pass YAML parameters automatically
